Journal of Grid Computing (2024) 22:44
https://doi.org/10.1007/s10723-024-09760-8

RESEARCH

Micro Frontend Based Performance Improvement and Prediction for Microservices Using Machine Learning

Neha Kaushik · Harish Kumar · Vinay Raj

Received: 15 December 2023 / Accepted: 18 March 2024 / Published online: 16 April 2024
© The Author(s), under exclusive licence to Springer Nature B.V. 2024

Abstract Microservices has become a buzzword in industry as many large IT giants such as Amazon, Twitter, Uber, etc have started migrating their existing applications to this new style and few of them have started building their new applications with this style. Due to increasing user requirements and the need to add more business functionalities to the existing applications, the web applications designed using the microservices style also face a few performance challenges. Though this style has been successfully adopted in the design of large enterprise applications, still the applications face performance related issues. It is clear from the literature that most of the articles focus only on the backend microservices. To the best of our knowledge, there has been no solution proposed considering micro frontends along with the backend microservices. To improve the performance of the microservices based web applications, in this paper, a new framework for the design of web applications with micro frontends for frontend and microservices in the backend of the application is presented. To assess the proposed framework, an empirical investigation is performed to analyze the performance and it is found that the applications designed with micro frontends with microservices have performed better than the applications with monolithic frontends. Additionally, to predict the performance of microservices based applications, a machine learning model is proposed as machine learning has wide applications in software engineering related activities. The accuracy of the proposed model using different metrics is also presented.

Keywords Microservices · Performance · Microfrontends · Regression

Harish Kumar and Vinay Raj are both are contributed equally to this work

N. Kaushik (✉)· H. Kumar
Department of Computer Engineering, J.C. Bose University of Science & Technology, YMCA, Faridabad 121006, Haryana, India

N. Kaushik
e-mail: 20001902008@jcboseust.ac.in

H. Kumar
e-mail: htanwar@jcboseust.ac.in

Department of Computer Applications, National Institute of Technology Tiruchirappalli, Tiruchirappalli 620015, Tamil Nadu, India

V. Raj
e-mail: vinayraj@nitt.edu

1 Introduction

In recent years, increasing demand for independently developed and deployed web applications has led to a considerable increase in the use of distributed systems. In today's environment, everything must be available online and delivered promptly for business purposes. Distributed systems are essential in order to speed up service delivery through continuous integration and continuous development [31]. Service Oriented Archi-

tecture (SOA) and Microservices Architecture (MSA) are two architectural styles based on distributed systems. Initially, applications were designed based on the monolithic architectural style. Monolithic architecture based applications were easier to develop, test and deploy because of their single code base. But as the application size increases it becomes difficult to maintain and accommodate new changes to the application because of its single, large and complex code base [2–5]. To overcome these challenges SOA architectural style came into existence.

SOA provides guidelines for the development of large enterprise systems by the composition of segments of different functionalities. SOA based applications are based upon the principle of distributed systems and these applications are easier to manage and maintain in comparison to monolithic architecture. Apart from several benefits there are the issues such as scalability, central governance of services and deployment of services that developers were facing [32]. All these issues led to the transition to microservices architecture based applications.

MSA is an architectural style where an application is a collection of loosely coupled and independent services and each service is responsible for performing a single task [8]. Microservices architectural style is an extension of SOA i.e. it inhabits all the properties of SOA along with some additional properties. In microservices architecture based applications, different technologies can be used for developing each service, separate teams are assigned for the development of individual services hence teams can work in parallel on several microservices which reduces development time, since the services are loosely coupled they can be maintained and deployed independent of other services. The loosely coupled nature of microservices makes them easier to maintain and accommodate changes without affecting the whole application. Delegation of team responsibility, maintainability, scalability, DevOps support, fault tolerance and separation of software responsibilities are some of the additional benefits of microservices architecture [10]. Because of several advantages associated with microservices architecture style, it has gained the attention of researchers and industry experts [1].

Today, internet giants such as Netflix, eBay, LinkedIn, Soundcloud and Amazon have adopted microservices architecture style for application development and researchers are working to get a better understanding of this architectural style. The optimal granularity of a microservice [33], service availability [36], Quality of Service (QoS) of microservices [34,35], microservices management [37], deployment and scalability [38,39] are the issues on which researchers are focusing currently. Apart from several different kinds of challenges, performance monitoring, performance improvement, self-adaptability, and performance prediction are the most common concerns among the researchers and practitioners [11–14]. The performance of MSA based application can be measured in terms of the time it takes to respond to user requestss i.e. Response Time, the number of requests completed per unit time i.e. Throughput or the number of resources an application uses to serve the user request i.e. Resource usage. The lifecycle of a microservices application can be divided into two phases, developmental phase and the operational phase [19]. The performance of an application can be improved by working on both phases of the application. But to the best of our knowledge, several researchers have suggested improving the performance by focusing on the granularity of the microservice but in actuality there is a lack of work done in order to optimise the performance of a microservice architecture based application by focusing on its developmental phase.

There is still a scope where by making changes to the architecture of microservices architecture based web applications, their performance can be improved. Moreover, making changes during the early stage of the application development not only improves the performance of the web application but it will also be less complex and cost effective.

The paper resolved the existing gap by presenting a model implementing micro frontend following microservices in the backend. The existing approaches either focus on the operational level improvements or they use only micro fronted or microservices but not using both approaches together. To the best of our knowledge, none of the existing work has implemented and proved the use of micro frontend following microservices as a solution for performance improvement of microservices architecture based web applications. In the proposed model, micro frontend is used for developing the frontend of the web application and the microservices approach is used for developing the backend of the application. The whole application is now divided into small services where a service will be responsible for executing a single task. All the ser-

vices will be developed, deployed, debugged, evolved and maintained independently of other services. In the proposed model a single team will be responsible for developing the backend and frontend of the application. The communication between the frontend services and backend services will take place via the API Gateway or aggregation layer. API gateway will decide to which service the user request should be forwarded for processing on the server side.

Machine learning models have been widely applied in software engineering especially in predicting and improving the performance of services which in turn has produced promising benefits. In order to predict the performance of microservices based applications considering the number of services and complexity of the application, a regression model is constructed with the data used for training. The proposed regression model is also tested for estimating the accuracy of the model.

For a better understanding of the proposed model and to clearly state the motivation behind the research work, the following Research Questions (RQs) have been framed:

RQ 1: How the performance of MSA based applications can be improved.

RQ 2: Will the implementation of the proposed model help in improving the performance of the microservices application or not?

RQ 3: How machine learning can be used to predict the response time of MSA based applications.

For RQ1, a framework including micro frontend followed by microservices in the backend of the web application has been proposed. To the best of our knowledge, which is based on the literature survey and studies on recent works that have been done by the practitioners in the field of microservices, none has tried to integrate these two technologies for the improvement of the performance of microservices applications. In the proposed framework, micro frontend has been used for better handling of user requests and the frontend of the web application and microservices in backend are used for efficient deployment of the application which leads to efficient processing of the user requests on the server side. To answer RQ 2, an empirical analysis has been made in order to compare the results of the proposed model and the existing model. The results show how the proposed model improved the throughput of microservices based web applications. For RQ 3, machine learning has been used to predict the response time of the microservices application using the number of services and the number of requests submitted to the application. Hence the results that will be obtained from this phase of the proposed work can be used by researchers and practitioners at an early stage of application development to improve the performance of the microservices application.

To summarise, this paper makes the following contributions:

- Developed a model where micro frontend has been used for designing the user interface of the application and microservices have been used for designing the backend architecture of the same application.
- Conducted an empirical analysis to investigate the performance of the proposed model and compared it with existing monolithic frontend following microservices architecture based application.
- Proposed a machine learning based model to predict the response time of microservices architecture based applications which can be used for making performance improvement decisions.

The remaining part of the paper is structured as follows: The background of the proposed work including monolithic and microservices styles is presented in Section 2. Related work from the literature is discussed in Section 3. The proposed model and its empirical evaluation are presented in Sections 4 and 5 respectively. An approach to predict the response time of microservices using ML is proposed in Section 6. The threats to validity of the proposed work are presented in Sections 7 and 8 concludes the paper.

2 Background

The preliminaries that are required to understand the proposed model have been discussed in this section. The primary aim of this paper is to improve the performance of MSA based applications by using micro frontend instead of monolithic frontend. In this section, to ease the readers understanding, details of these different architectural styles are discussed.

2.1 Monolithic Architecture

Monolithic architecture is a single-tiered architecture where different components are combined into a single

program running as a single deployment process [2]. Monolithic architecture based applications are easier to develop, test and deploy because of a single code base [4]. However, the encapsulation of all components as a unified process results in a tightly coupled software application where the modules cannot be executed independently and this problem becomes more complex as the size of application increases [3,5]. Because of the tightly coupled nature of monolithic applications, changes in one component can affect the working of other components and because of the complex and large code base, it is difficult to debug, evolve and maintain the whole application [6]. The coordination of the development team also becomes difficult as the number of developers are high in monolithic applications [7]. Long term commitment to a technology stack, continuous deployment and scaling the application are some of the other drawbacks associated with monolithic architecture [4]. Because of these drawbacks, organisations have migrated to microservices architecture style.

2.2 Microservices Architecture

The concept of microservices architecture was first discussed by James Lewis and Martin Fowler. Microservices Architecture (MSA) is based upon the distributed system approach where the web applications are divided into loosely coupled, independent, highly cohesive and finely grained services. Each microservice is responsible for performing only a single task representing a business capability and uses lightweight mechanisms such as HTTP and REST APIs for communicating with other microservices [8,9]. MSA enables each microservice to be developed, tested, deployed, and upgraded independently [30]. Microservices architecture provides independent services only in the backend of the application. Frontend of the application still faces a significant degree of dependency and this dependency leads to coupling and communication overhead [15].

2.3 Micro Frontend

In recent years, the concept of micro frontend has been used for developing the front end of web applications. Micro frontend is an approach where the frontend of the application is divided into small services. Each service can be designed by a separate team using different technologies. In micro frontend architecture, the routing layer plays a key role in managing the routing and navigation of each micro frontend module as well as the overall application. It achieves this by intercepting user requests and determining which micro frontend module should handle the request based on the requested Uniform Resource Locator (URL). When a user makes a request, the routing layer checks if the requested URL matches any of the registered routes. If a match is found, the routing layer identifies the micro frontend module responsible for handling that route and forwards the request to it. The micro frontend module then retrieves the necessary data and renders the appropriate view or component, which is then displayed to the user. If no match is found for the requested URL, the routing layer can either display a default page or redirect the user to an error page. Technology agnostic, code isolation, naming conventions and resilient websites are some of the additional benefits of the micro frontend approach to software development [18]. Micro frontend architecture based applications have independent services in the frontend of the application but the monolithic backend of the application still faces dependency.

Figure 1 depicts the architectural difference between monolithic architecture, MSA and micro frontend based applications. In the below figure, it is clearly visible that the frontend and backend or the monolithic architecture based application has a single code base. This indicates that the whole application including the backend and frontend will be developed by several teams using the same technology stack, then integrated into a single code base and then deployed on the cloud. Now in case of heavy workload several instances of the whole application will be created, which will be costly in terms of resource consumption. The second disadvantage of monolithic architecture is that if you want to make changes in one module of the application then the whole application will be unavailable to the user because of a single code base. Now if you see microservices architecture in Figure 1, you will notice that the application is divided into several small services (microservices) in the backend. It means in case of heavy workload on a microservice multiple instances of that particular microservice will be created while

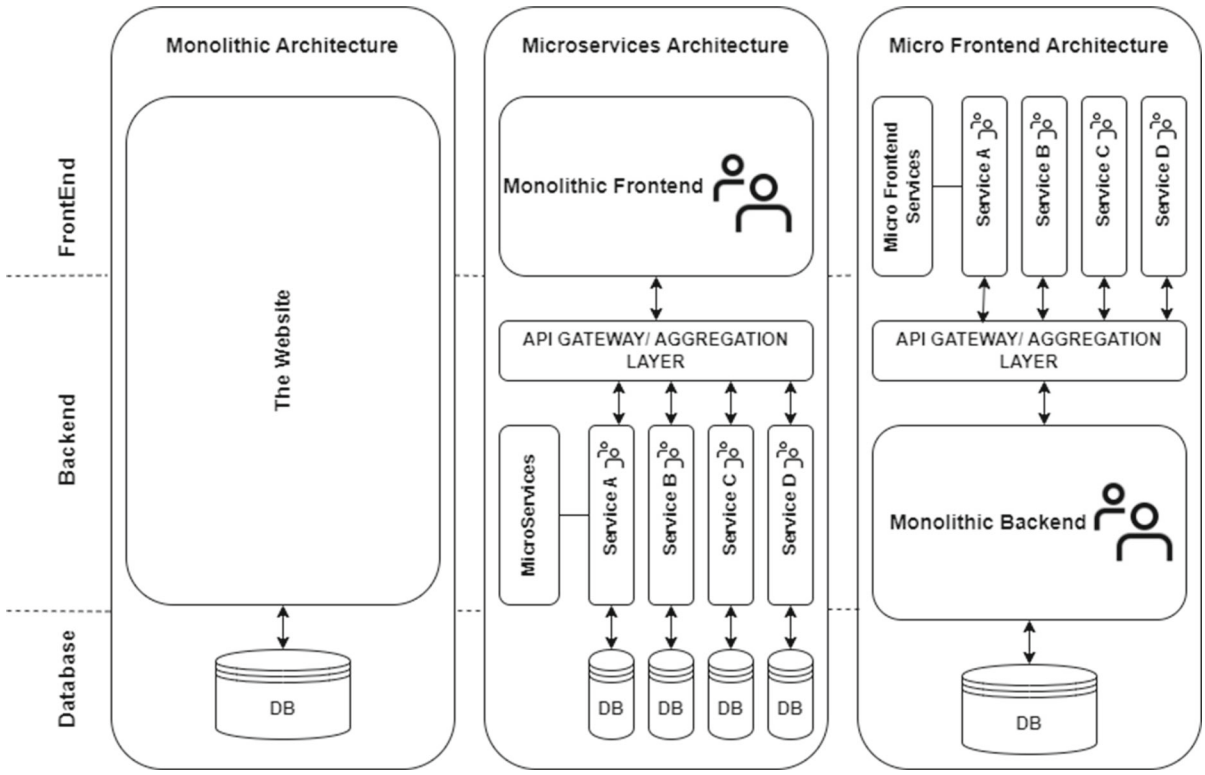


Fig. 1 Comparison of Several Application Development Architectures

keeping the instances of other services as it is. So it will be resource and cost efficient. But now if we see the frontend of an MSA based application in Figure 1 we can observe that it has a monolithic frontend, which means if we try to make changes in the frontend then again the whole application will be impacted. Micro frontend in Figure 1 has independent services in the frontend of the application, which indicates that making changes in frontend would be easier and when the user sends a request, its request will be forwarded to the appropriate service by API gateway but in the backend it still has monolithic architecture which makes it difficult to scale and deploy.

Hence to fill the gaps observed in the above discussion, a novel framework has been proposed, where the web application has a micro frontend following microservices in the backend. Micro frontend efficiently improves the handling and forwarding of user requests and microservices in the backend help in the proper processing of requests and proper handling of heavy workloads with efficient utilisation of resources. The performance of the proposed framework is also compared with the existing model and results are presented in the paper.

3 Related Work

3.1 Studies on Performance of Microservices Architecture

The deployment complexities of microservices increase as they scale. It becomes important to measure and monitor their performance before they degrade the performance of the whole system. Brondlin and Santambrogio [40] proposed a Black-Box monitoring approach for microservices deployed with Docker containers and managed by a Kubernetes orchestrator. The author has used performance counters, network performance, power consumption and resource usage metrics to monitor the microservices at scale. However, the focus of this work is on data monitoring, which has its limitations in terms of how and where this monitored data may be used to predict or opti-

mise microservice performance. Abdullah et al. [11] proposed a burst aware autoscaling method to solve the problem of detecting and handling burst workload in containerized microservices based applications in order to preserve performance. The proposed model resulted in performance improvement but the limitation of the proposed model is that it was tested on a single microservice but not on the application containing multiple microservices. The distributed nature of microservices makes resource management a concern as it not only increases the cost but also affects the performance of application. Joseph and Chandrasekaran [19] reduced the response time of microservices and performed efficient utilisation of resources by deploying the microservices based on their interaction with other microservices. However the author focused on the efficient utilisation of resources but not consider the impact of network characteristics such as link congestion before making deployment decisions. Additionally, energy efficiency of nodes is also a key point to consider in deployment decisions, which was ignored in the proposed model of the author. Sampaio Jr. et al. [20] has used a three step approach for optimising the performance of microservices application. The first step includes the monitoring of microservices to obtain resource, communication affinities related information. Second step is to identify the performance oriented optimal place for locating the microservices. The third step is the migration of microservices based on the information obtained from previous steps. The approach used by the author sometimes improves the performance but at the same time in some cases it is also responsible for performance degradation. Camilli and Russo [21] focused on identifying the microservices that can lead to performance degradation of complete applications. The results of the proposed model can be used by the engineers during system maintenance. Cortellessa et al. [22] monitors the runtime interactions of all microservices of an application and then uses the monitored data to improve the performance by re-engineering the design of the application. In the proposed approach, the refactoring decisions are fully automated, and it is not a good practice to neglect human expertise in software re-engineering. Luo et al. [23] have done an in depth study of microservices call graphs to find the possible solutions for optimising the performance.

3.2 Studies on the Role of Machine Learning in Performance of Microservices

Abdullah et al. [11] has performed resource prediction using the Decision Tree Regression technique and after resource prediction the author has predicted the workload using Elastic Net (EN) regression model. Based on the output received from these two steps the autoscaling decisions are made. The proposed algorithm succeeds to handle bursty workload situations while keeping less response time. The drawback of the proposed model is that the author has used a single microservice in its experimentation. Hence whether the proposed approach is suitable for the whole application including several microservices is still a concern. Qassem et al. [26] has used a proactive approach for making autoscaling decisions. Random forest model has been used to predict the future memory and CPU requirements of each microservices. Later based on the predicted values horizontal and vertical scaling decisions are made. In future progressive learning can be used to get better and more accurate results. Yan et al. [27] used Bi-LSTM for predicting the workload and performed elastic scaling using reinforcement learning algorithms. The load prediction model used by the author is inaccurate and requires update and regular training as the business environment changes. Xu et al. [28] have proposed CoScal using gradient descent units for predicting the workload in time series manner and reinforcement learning algorithms for efficient scaling decisions. The proposed model succeeds in reducing the response time by 6-10%. Deep learning can be used in future for improving the accuracy of workload prediction.

3.3 Studies on Micro Frontend

The author Peltonen et al. [17] has discussed the benefits and issues related to Micro Frontend architectural style. The author in the survey found that the Micro frontend is a new architectural style with several benefits and lack of necessary functionalities that can support Micro Frontend. Yang et al. [24] has designed a content management system using micro frontend architectural style and compared it with the monolithic frontend style. The author found that it is less complex

to manage the system by using micro frontend in comparison to monolithic frontend as the application size grows. Prajwal et al. [25] have studied the pros and cons associated with the micro frontend architectural style. Technology agnostic, fault isolation, team independence and release independence are some of the benefits associated with Micro Frontend while redundancy, communication and risk of code divergence are some of the issues that need to be addressed. Wang et al. [16] have designed a graduate informed system using micro frontend architecture. The author used micro frontend architecture to verify how well this architecture minimizes coupling and costs while improving the composability and scalability of the system.

Joseph et al. [19] divided the lifecycle of microservices application into developmental phase and operational phase. Based on the analysis of the first study, it is found that most of the researchers have been focusing on the performance improvement of microservices application by working on the operational stage. None of the existing work has focused on performance improvement by working in the development phase of the application. Making improvements in the early stage is cost effective and easier to do. The second study shows how proactive actions are better than reactive actions. The use of Machine Learning leads to a proactive action which eliminates the chance of performance degradation. The third study of the literature shows that Micro Frontend is a new architectural style with several benefits and challenges that need to be addressed. To overcome all these existing gaps, a model is proposed to predict and improve the performance of MSA based applications.

4 Proposed Model

In this section, an answer to RQ1 has been provided by having a detailed discussion about the proposed model. In the proposed model, micro frontend following microservices in the backend has been used. We will design an application, where the whole application will be divided into small and independent services. Each service will be responsible for performing a single task and will have access to its separate database. Every webpage of the application can consist of a single or multiple services depending upon its requirement. A single team will be responsible for the designing the frontend, backend and maintenance of the service. All the services can be developed using different programming languages. When using micro frontends and microservices together, the front-end modules are developed and deployed independently, but they still need to communicate with the back-end services. This can be done using well-defined APIs and a gateway that handles the communication between the front-end modules and the back-end services. Figure 2 depicts the proposed architecture, where an application is divided into four services named as Service A, Service B, Service C and Service D. A real time application can have any number of services depending upon the requirement but here so better understanding of the proposed model we have only four services. The backend, frontend of each service will be developed by separate teams. Each service can be developed using different technologies in the backend and frontend and different teams can work on their services simultane-

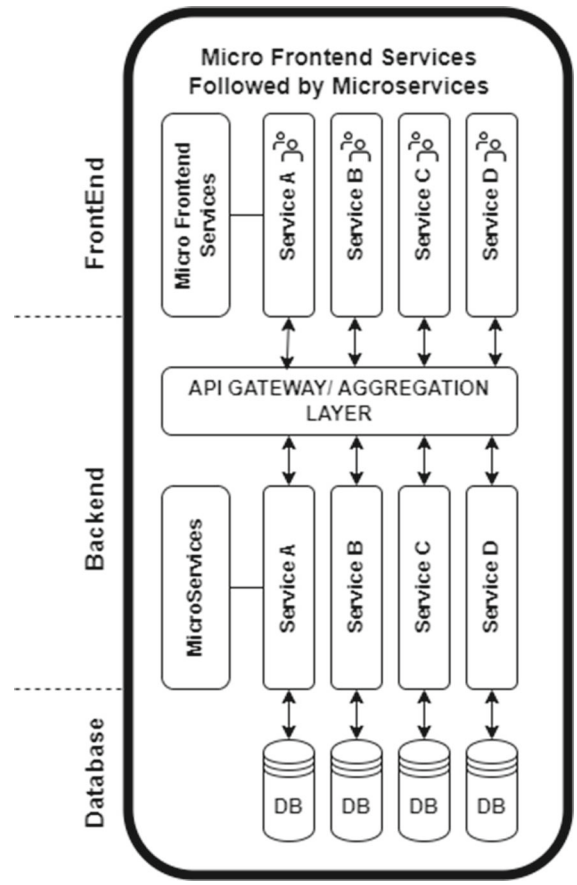


Fig. 2 Architecture of micro frontend and microservices

ously hence reducing development time of the application. Each service has its own database and can be deployed in the cloud independent of other services. In case of heavy workload on a particular service, there is no need to increase the instances of all the services, only the instance of that particular service will be increased which in turn reduces cost and improves efficient utilisation of resources.

Figure 3 shows the working of the proposed model. When a user clicks on a link, the routing layer is responsible for identifying the micro frontend service responsible for handling the user request. Routing layer will use the URL of the user request in order to identify the appropriate frontend service for handling the user request. After frontend service identification the service displays the appropriate interface and options to the user. Now the user will interact with the micro frontend and micro frontend response to the user based upon its clicks. Now if the user sends some request to the server then the frontend service forwards that request to API gateway and now API gateway is responsible for forwarding the user request to the right backed microservice. The microservice processes the user request using its database(if needed) and if the microservice wants to communicate to other microservices then it communicates with other microservices through the API gateway only. Now after processing the user request, the microservice sends the response to the API gateway which in turn forwards the response to appropriate frontend service. After receiving the response the frontend service displays the appropriate output to the user.

The division of whole application into individual services make it easier to design and develop it, allow faster delivery of the application to the client, loosely coupled services require less communication hence reducing the communication overhead and improving response time, making changes to one application does not affect the other services and allows the other services to work. Hence it is easier to maintain the application developed using proposed model, independent services makes the deployment and scaling operations easy to implement and cost effective.

5 Empirical Analysis

In this section we have analysed the performance of the proposed model and the existing approach in order to answer RQ2. RQ2 focused on the implementation of the proposed model and tried to find whether or not the proposed model succeeded in improving the performance of the microservices application or not.

5.1 Case Study Application

To perform the empirical analysis, an Eshop application using monolithic frontend followed by microservices in the backend and micro frontend followed by microservices in the backend is developed using the server-side template composition. It also uses container application server to manage all the services on a single web page. Figure 4 represents the case study application developed using a monolithic frontend. The frontend of this

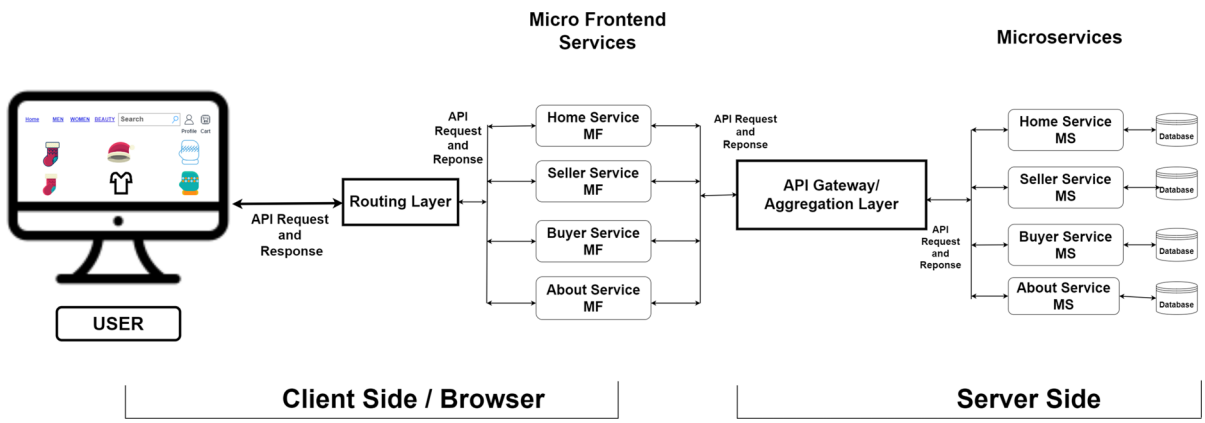


Fig. 3 Working of micro frontend followed by microservices

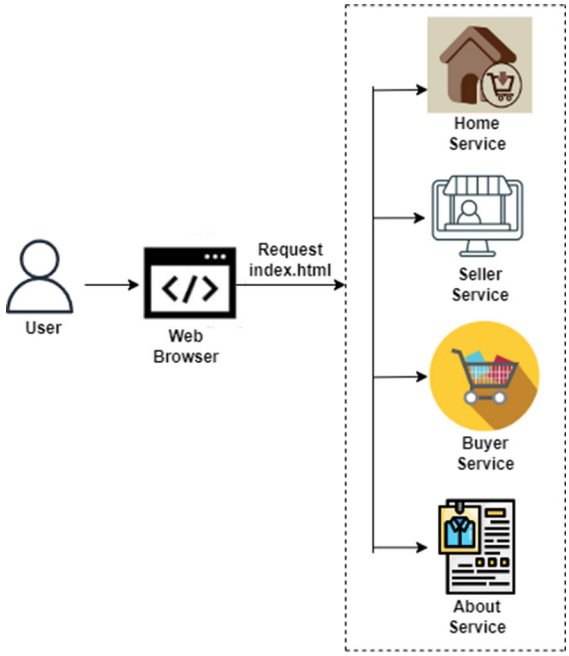


Fig. 4 Case Study Application using Monolithic Frontend

application has four main components as Home service, Seller Service, Buyer service and About Service. Same technology stack has been used for developing these components of the front end. These components are then integrated to form a single code base. The backend of this application has four different microservices named as Home service, Seller Service, Buyer Service and About Service. These services are developed using different technology stacks and are independent of each other. When the user accesses this application, the web browser firstly downloads the frontend code of the application and then handles and renders the user interface on the computer screen based on user actions. While on the server side, when the browser sends HTTP request to the server, the server receives that HTTP request and then calls the API gateway. API gateway further calls the appropriate service to handle the user requests. In Figure 4, Home service represents Home component which is responsible to display the home page of the application, Seller service represents the seller component of the application which is responsible for displaying the information related to all sellers and their ratings, Buyer service representing the buyer component which is responsible for displaying information related to each buyer and their past shopping details. About Service represents the About component which is responsible for displaying information of all the products available on the website. After developing all these components they are integrated into a single and integrated codebase to form an application. The backend services are responsible for maintaining the information related to each component.

Figure 5 represents the case study application having micro frontend and microservices in the backend. To build this application, the whole application is divided into four services that will be responsible to perform a single task and independent of each other. These four services are named Home service, Seller service, Buyer Service and About service. The task of home service is to display and maintain the information related to the Home page of the application. The Seller service will be responsible for maintaining and displaying the information related to each seller and their rating given to them by the customers. The Buyer service will maintain and display the information related to each buyer and their previous purchase history. The About service will be responsible to maintain and display the information related to each product available online. Now the difference in this application in comparison to the previous application is that there will be a separate team for each service which will develop the frontend and backend of their assigned service using different or same technology. These services will be deployed independent of other services and in case of updates to one service other services will keep working and remain unaffected. When the user accesses the application then the browser will call the container application server, which will call and assemble the respective services that are needed to be displayed on the page based on user action. A single page will contain a single or more number of services.

5.2 Empirical Analysis

In order to validate the proposed approach of integrating micro frontend with microservices, it is required to perform empirical analysis of the proposed approach with existing standard of designing web applications. In this section, empirical analysis performed with respect to performance parameters i.e, response time and throughput is analyzed and results are presented.

To perform the empirical analysis, the main aim was to capture the response time(s), throughput, and error percentage by providing varying loads from 1 to 10000

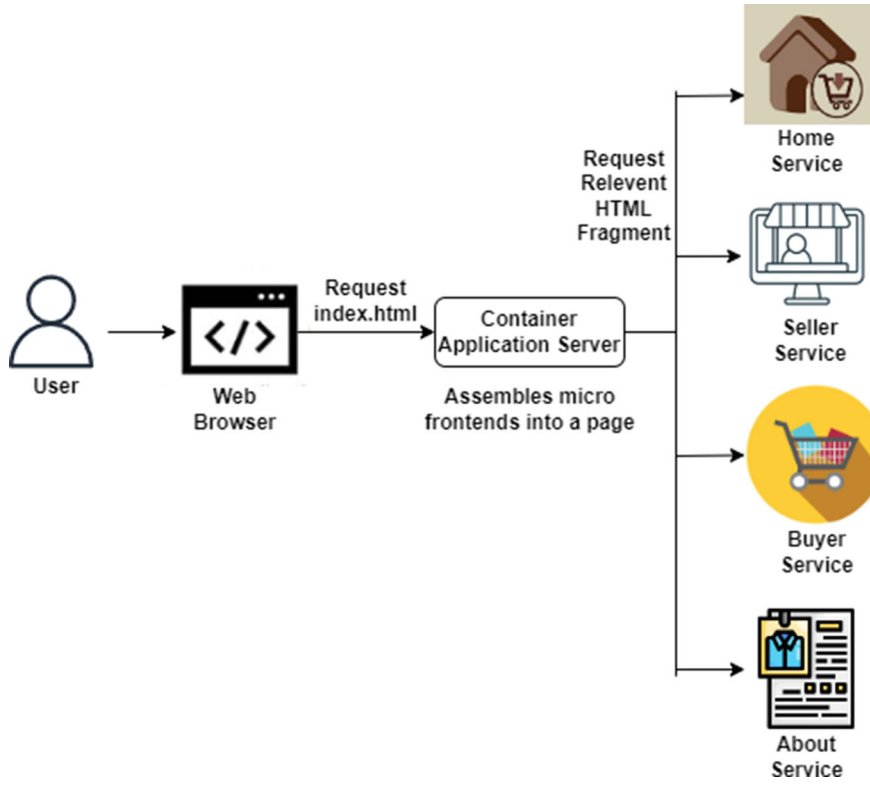


Fig. 5 Case Study Application using Micro Frontend

user requests. To achieve it, Apache JMeter tool is used in our work which helps in accessing the performance by conducting load testing. The case study application receives user requests from the tool with different loads by hitting requests through HTTP and the results are captured for varying loads, which are presented in Tables 1 to 4.

The tables in this section mainly have seven fields. Architectural style field represents the existing style used to develop microservices applications and architectural style that has been used in the proposed model. In this work, monolithic frontend followed by microservices architecture in the backend has been used as an existing approach of developing MSA applications and for the proposed model micro frontend followed by MSA in backend has been used. So the Architectural style field of the tables will represent these two categories only. Second field of the tables is Number of users, it represents the number of users that can send HTTP requests to the application simultaneously. Average response time is the average time taken to respond to the user request. Maximum response time represents the maximum time the application takes to respond to the user request. Minimum response time represents the minimum time an application takes to respond to the user request. All response times are measured in milliseconds. Error percentage represents the number of failed requests out of total requests that were sent to the application. Throughput represents the number of user requests completed per second.

Table 1 represents the data for the Home Microservice of the application. In this table, the number of user requests vary from 1 to 10000. The table also represents response time, throughput and error rate using micro frontend and without using micro frontend. We can see from the table that the proposed model leads to decrease the response time of the microservice as the number of users is increasing. The proposed model leads to increasing the throughput of the home microservice in comparison to existing approaches. However, it is also observed that the error percentage is increasing when the number of users exceed 5000. From the table, it is observed that the error percentage for both the styles is 30.74% and 51.47% respectively. As Jmeter has been used for testing the performance, a maximum of 10000 user requests can be sent to the application and it is the main reason for the failure of requests when hitting with 10000 requests. Therefore, it does not reflects

Table 1 Performance Table of Home Service

Architectural Style	Number of Users	Average Response Time (in ms)	Minimum Response Time (in ms)	Maximum Response Time (in ms)	Error Percentage	Throughput
Monolithic Frontend followed by MSA in backend	1	2	2	2	0.00%	500.0/sec
Micro frontend followed by MSA in backend	1	2	2	2	0.00%	500.0/sec
Monolithic Frontend followed by MSA in backend	10	4	2	6	0.00%	3.7/sec
Micro frontend followed by MSA in backend	10	1	1	3	0.00%	3.7/sec
Monolithic Frontend followed by MSA in backend	100	3	1	9	0.00%	33.8/sec
Micro frontend followed by MSA in backend	100	3	1	8	0.00%	33.8/sec
Monolithic Frontend followed by MSA in backend	1000	1	0	11	0.00%	334.0/sec
Micro frontend followed by MSA in backend	1000	1	0	6	0.00%	334.4/sec
Monolithic Frontend followed by MSA in backend	5000	477	3	4092	0.16%	784.1/sec
Micro frontend followed by MSA in backend	5000	8	0	74	0.00%	1001.2/sec
Monolithic Frontend followed by MSA in backend	10000	2741	7	5997	30.74%	939.8/sec
Micro frontend followed by MSA in backend	10000	1671	2	6492	51.47%	1095.4/sec

Table 2 Performance Table of Seller Service

Architectural Style	Number of Users	Average Response Time (in ms)	Minimum Response Time (in ms)	Maximum Response Time (in ms)	Error Percentage	Throughput
Monolithic Frontend followed by MSA in backend	1	2	2	2	0.00%	500.0/sec
Micro frontend followed by MSA in backend	1	1	1	1	0.00%	1000.0/sec
Monolithic Frontend followed by MSA in backend	100	6	1	100	0.00%	33.7/sec
Micro frontend followed by MSA in backend	100	3	1	6	0.00%	33.6/sec
Monolithic Frontend followed by MSA in backend	1000	1	1	13	0.00%	333.7/sec
Micro frontend followed by MSA in backend	1000	1	1	11	0.00%	333.9/sec
Monolithic Frontend followed by MSA in backend	5000	125	1	774	0.00%	995.6/sec
Micro frontend followed by MSA in backend	5000	55	1	249	0.00%	1001.2/sec
Monolithic Frontend followed by MSA in backend	10000	3380	2	8997	73.02%	917.8/sec
Micro frontend followed by MSA in backend	10000	2504	2	6294	51.14%	1237.2/sec

Table 3 Performance Table of Buyer Service

Architectural Style	Number of Users	Average Response Time (in ms)	Minimum Response Time (in ms)	Maximum Response Time (in ms)	Error Percentage	Throughput
Monolithic Frontend followed by MSA in backend	1	2	2	2	0.00%	500.0/sec
Micro frontend followed by MSA in backend	1	1	1	1	0.00%	1000.0/sec
Monolithic Frontend followed by MSA in backend	100	5	1	10	0.00%	33.5/sec
Micro frontend followed by MSA in backend	100	4	1	8	0.00%	33.8/sec
Monolithic Frontend followed by MSA in backend	1000	1	1	8	0.00%	332.1/sec
Micro frontend followed by MSA in backend	1000	1	1	7	0.00%	334.2/sec
Monolithic Frontend followed by MSA in backend	5000	63	1	241	0.00%	1002.0/sec
Micro frontend followed by MSA in backend	5000	19	1	139	0.00%	1003.0/sec
Monolithic Frontend followed by MSA in backend	10000	2651	2	5546	72.58%	1406.3/sec
Micro frontend followed by MSA in backend	10000	3094	2	5378	50.37%	1457.5/sec

Table 4 Performance Table of About Service

Architectural Style	Number of Users	Average Response Time (in ms)	Minimum Response Time (in ms)	Maximum Response Time (in ms)	Error Percentage	Throughput
Monolithic Frontend followed by MSA in backend	1	3	3	3	0.00%	333.33/sec
Micro frontend followed by MSA in backend	1	2	2	2	0.00%	500.0/sec
Monolithic Frontend followed by MSA in backend	100	5	1	9	0.00%	33.6/sec
Micro frontend followed by MSA in backend	100	4	1	9	0.00%	33.7/sec
Monolithic Frontend followed by MSA in backend	1000	1	1	12	0.00%	333.0/sec
Micro frontend followed by MSA in backend	1000	1	1	5	0.00%	334.4/sec
Monolithic Frontend followed by MSA in backend	5000	93	1	1125	0.00%	997.6/sec
Micro frontend followed by MSA in backend	5000	51	1	239	0.00%	1001.2/sec
Monolithic Frontend followed by MSA in backend	10000	4229	6	8985	69.40%	847.0/sec
Micro frontend followed by MSA in backend	10000	2875	2	6209	33.93%	1237.6/sec

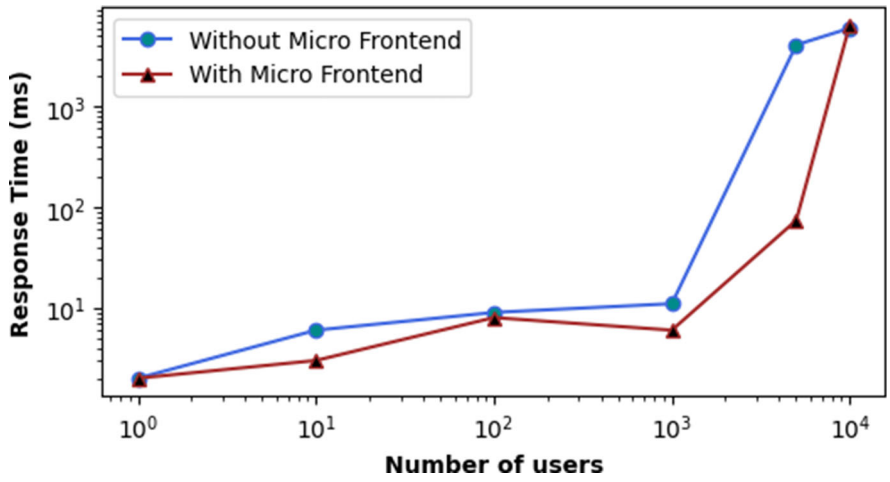


Fig. 6 Maximum Response Time of Home Service

any issue with the proposed approach or the chosen case study application. However, some advanced testing tools such as K6, Locust, ApacheBench can be used for testing but these tools do require higher configuration systems for processing the requests.

The maximum response time of the application using a micro frontend, followed by the MSA and the application without using a micro frontend, is shown graphically in Figure 6. From Figure 6, it is clear that the use of a micro frontend with a microservice performs better than the existing approach since it reduces the response time of the application.

Figure 7 is the graphical representation of the average time taken to respond to user requests by the proposed model and the existing model. Figure 7 shows the use of micro frontend results in lesser response time than the application which is not using micro frontend. The maximum response time and average response time represent the worst case and average case scenarios of time complexity. So in order to analyse the performance of the proposed model and existing model the maximum and average response time matrices are considered. The results show that in the case of the Home service proposed model performs better than the existing model.

Table 2 represents the performance matrix of Seller service. We can see how the response time using the micro frontend and microservices decreases in comparison to monolithic frontend followed by microservices. The gap between the response times using both the approaches is increasing as the number of users is increasing. The error rate is also less when we are

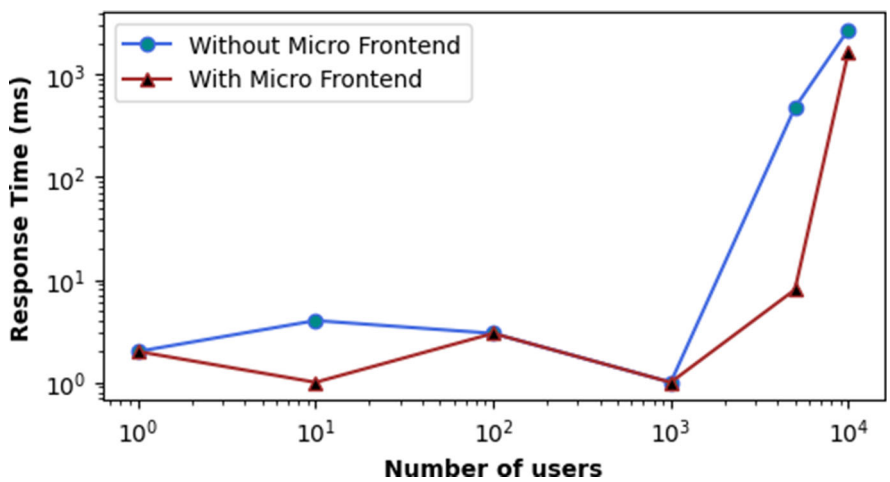


Fig. 7 Average Response Time of Home Service

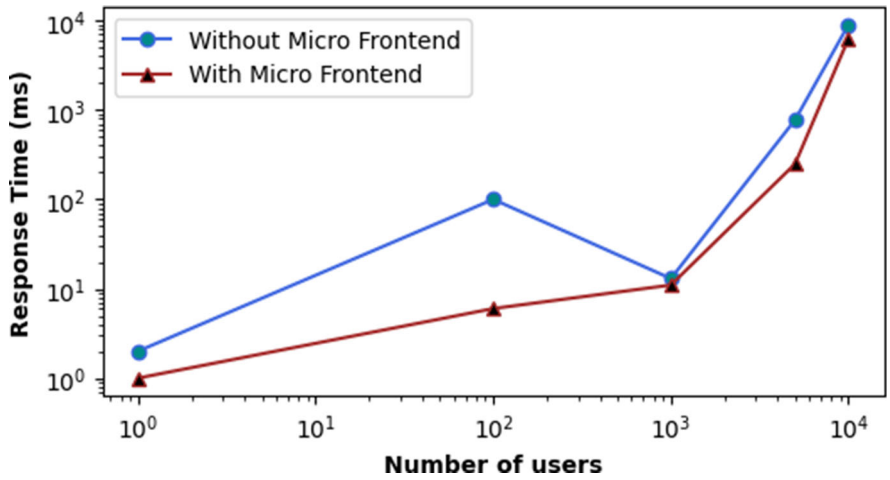


Fig. 8 Maximum Response Time of Seller Service

using the micro frontend. The number of requests successfully executed per second is also larger when we use the micro frontend.

Figure 8 is the graphical representation of the maximum response time of the Seller service. It represents the maximum response time of the proposed model which uses micro frontend to improve the performance of the application and existing model which do not use micro frontend. The graphical representation shows that the proposed model is more efficient than the existing model in terms of performance.

Figure 9 represents the average response time of Seller service with and without micro frontend. From this figure, it is clear that the proposed model helps in reducing the average response time in most of the cases. Hence we can say that the proposed model works better than the existing model in the case of Seller service.

Table 3 represents the performance of Buyer service using both micro frontend and monolithic frontend. We can see from the table that whether the number of users is 1, 10, 100, 1000, 5000 or 10000, for every number of users the error rate and response time of micro frontend based application is either the same as existing approach or it is decreased and the throughput is also increased using the micro frontend.

Figure 10 shows the graphical representation of the maximum response time of Buyer service. As we can see, the use of a micro frontend either reduces the response time or keeps it the same as that of the proposed model. So we can say that in the case of Buyer

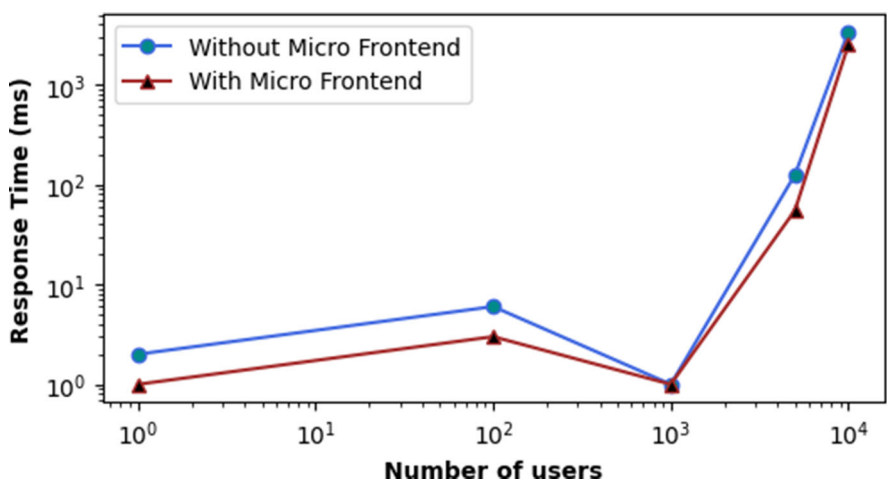


Fig. 9 Average Response Time of Seller Service

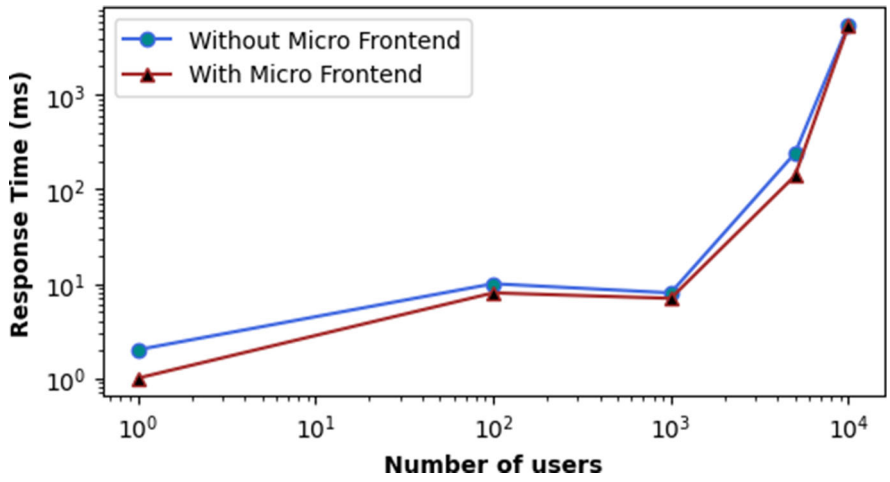


Fig. 10 Maximum Response Time of Buyer Service

service, the use of a micro frontend performs better than existing model as it reduces the response time in some cases.

Figure 11 represents the average response time of proposed model and existing model. The use of the proposed model reduced the response time in all cases except when the number of users is 10000.

Table 4 represents the response time, error rate and throughput of About service using micro frontend and without micro frontend. The results show that the micro frontend is giving better results than the monolithic frontend. No matter what the number of users, in all the cases micro frontend and microservices are giving better results. And as the number of users is increasing the gap between the results is also increasing and making the proposed approach work better.

Figure 12 depicts the graphical representation of the response time collected during implementation of proposed model and existing model. This figure shows that for About service, the use of micro frontend leads in reducing the response time for any number of users. It means we can say that for About service the proposed model of micro fronted and microservices performs better than the existing model.

Figure 13 is the graphical representation of the Average response time of the About service of the case study applications. The graph shows that the average response time of about service is either reduced in most of the cases or remains the same. Hence we can say that for About service, the proposed model leads to better performance in comparison to the existing model which uses monolithic frontend and microservices.

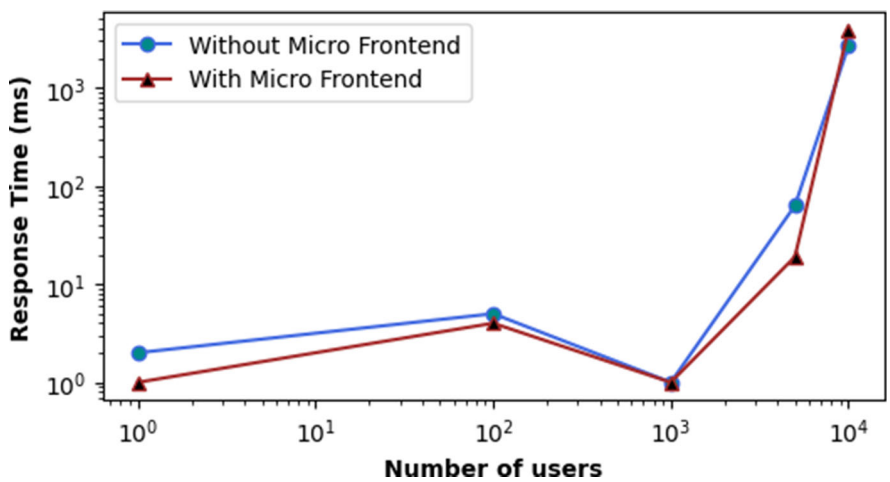


Fig. 11 Average Response Time of Buyer Service

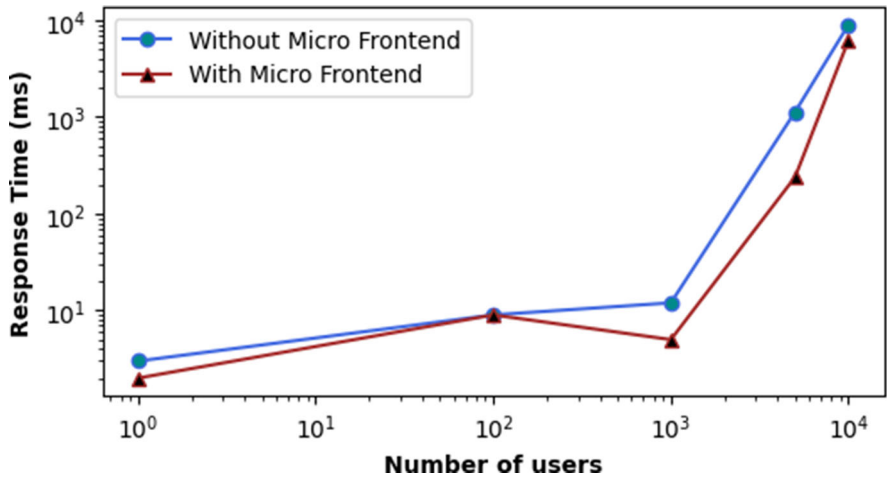


Fig. 12 Maximum Response Time of About Service

After observing the maximum and average response time of all the services using the proposed model i.e. micro frontend followed by MSA and existing model i.e. monolithic frontend followed by MSA, we can say that proposed model leads to reduced response time. Also, the throughput of all the services with the number of users as 1, 100, 1000, 5000 and 10000 is graphically represented in Figures 14 to 18 respectively. The graph shows that the proposed model lead to improve throughput in most cases.

5.3 Discussion

After analyzing the response time and throughput of the proposed approach with respect to the earlier styles of designing applications, it is clear that the proposed approach is definitely a recommendation for designing large enterprise web applications. The summary of our findings is discussed here.

The findings indicate that the average response time using a Micro Frontend is typically better than the average response time obtained using a Monolithic Frontend for all services and also for varying numbers of users. In some cases, the average response time of micro frontend architectural style is the same as that of monolithic frontend. For example, when the number of users is 1000, all the services have the same average response time for both the architectural styles i.e. using micro frontend and without micro frontend. In Figure 7, when the number of users is 100, Home service has the same average response time for both the styles. Only in one case the average response time of the micro frontend based architectural style is higher than the average

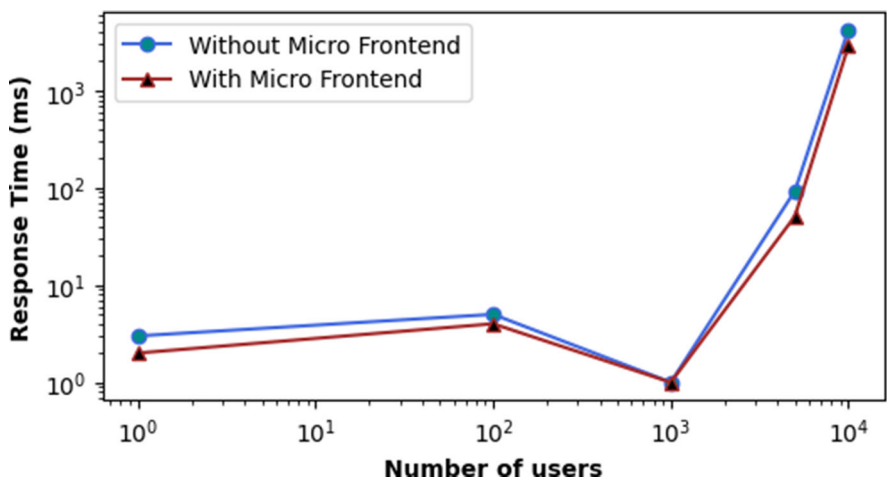


Fig. 13 Average Response Time of About Service

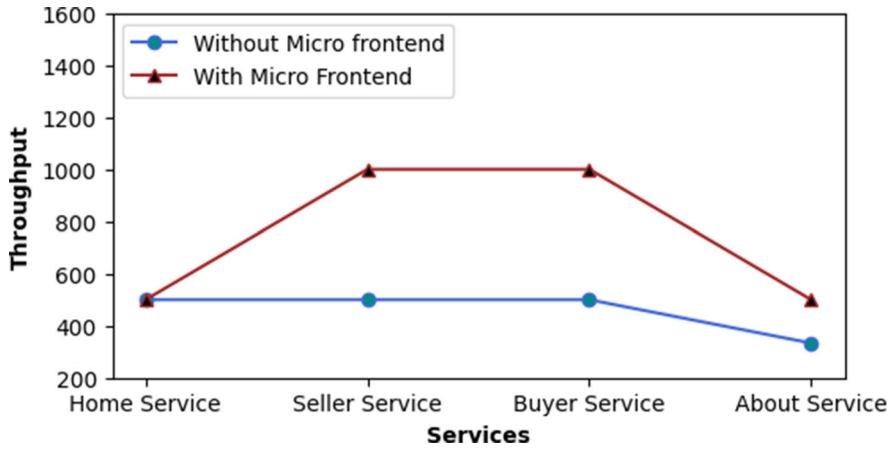


Fig. 14 Throughput when the number of users is 1

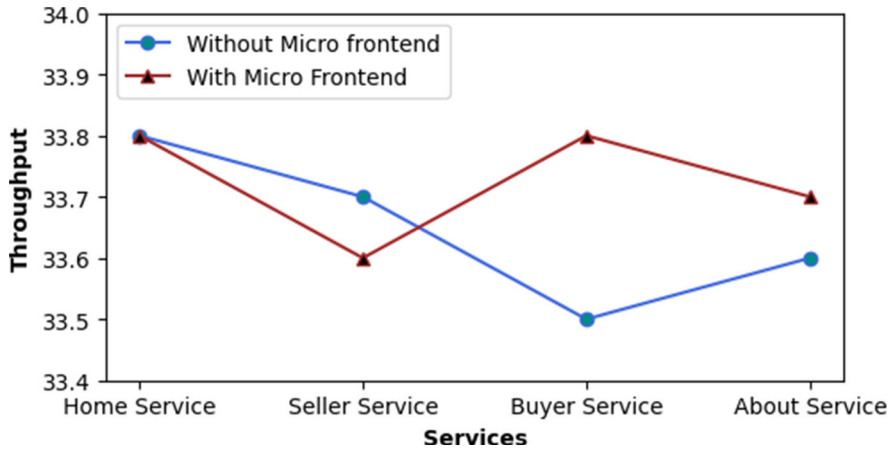


Fig. 15 Throughput when the number of users is 100

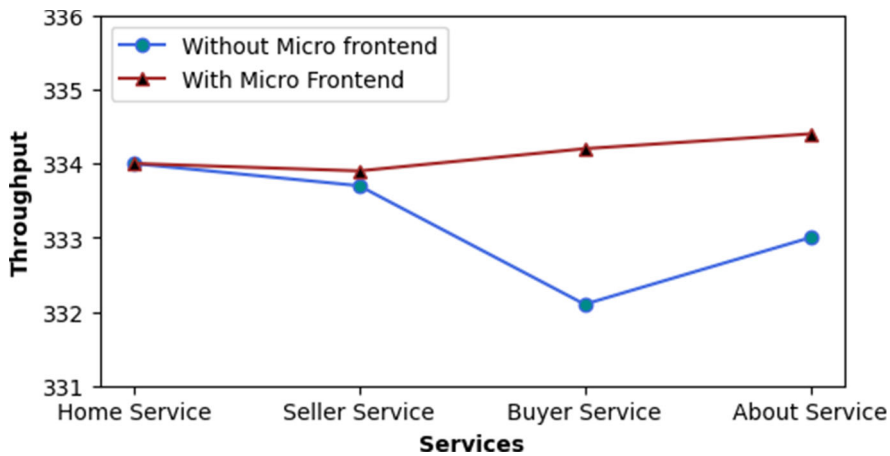


Fig. 16 Throughput when the number of users is 1000

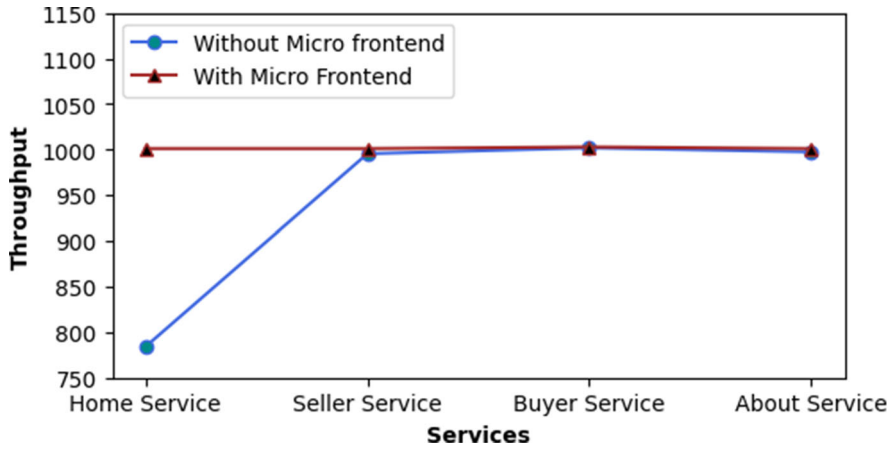


Fig. 17 Throughput when the number of users is 5000

response time of the monolithic frontend. For example, the Buyer service using micro frontend has high average response time when the number of users is 10000. The maximum response time using micro frontend is always less than the maximum response time without micro frontend. Only in two cases, the maximum response time is the same for both architectural styles. For example in Table 1, when the number of users is 1, and in Table 4 when the number of users is 100, the maximum response time for monolithic frontend and micro frontend is the same. Figure 14 to 18 represent the service wise comparison of throughput for different number of users. The figures show that in most of the cases, the proposed framework leads to better throughput than the existing approach. The throughput of Home service is same for both architectural styles when the number of users is 1, 100 and 1000.

The comparison between both the approaches is based on the response time, error percentage and throughput matrix. And in all three matrices, frontend based applications lead to reduced response time i.e. faster response to the end user for their HTTP requests, increased the throughput and reduced number of failed requests. Observing the performance matrix of all the microservices and the graphical representation of response time and throughput, it can be concluded that the proposed model is giving better results than the existing model.

6 Performance Prediction

In this section, the regression model for predicting the performance of microservices based applications, the dataset information about the microservices applica-

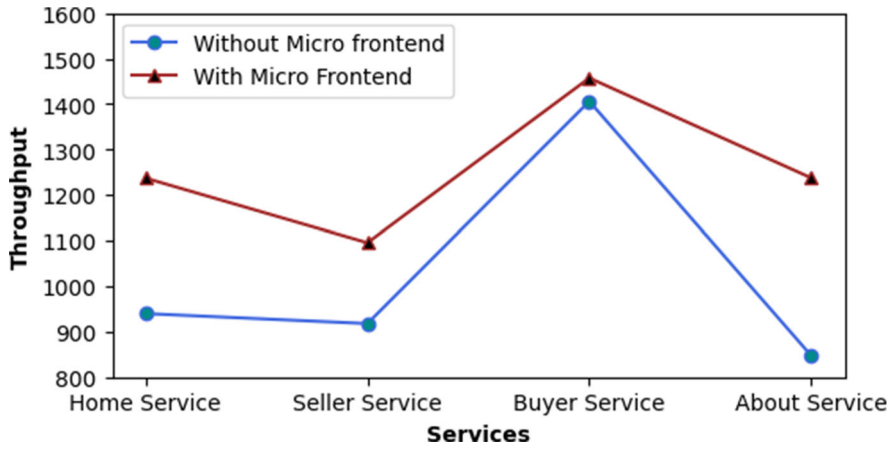


Fig. 18 Throughput when the number of users is 10000

tions and the measures to evaluate the accuracy of the proposed regression models are presented.

6.1 Regression Analysis

Regression is one of the important methods of analyzing the relationship between dependent and independent variables by generating a model to present the relationship. If the number of independent variables is more than two, then the generated model is called as multiple regression and it is most preferred and applied method in software engineering analytics. As the aim of this proposed model is to estimate the performance of a microservices based application, it is dependent on many factors including the number of microservices and number of requests submitted to the application. We define a multiple regression model based performance prediction model which is represented based on the below equation.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \epsilon \tag{1}$$

where y represents the performance, x_1 represents the number of microservices in the application and x_2 represents the number of user requests given to the microservices application. The values of x_1 and x_2 are considered as the independent variables and the coefficients β_1, β_2 and β_0 represent the constant values. Sometimes during the calculation, error is induced, which is represented using ϵ.

6.2 Dataset

The collection of dataset for microservices based application is really a challenge due to the inability to access projects developed in industry and the unavailability of performance metrics of the applications available in public datasets. Due to this, applications considered as case studies in different research articles from literature are considered as dataset. The required information for this regression analysis is gathered from the papers and the details of which are presented in Table 5. Total of 10 microservices based application data are collected, out of which 8 records are used for training the model and the remaining two applications for testing the constructed regression model.

6.3 Evaluation Criteria

To evaluate the accuracy of the prediction model, several frequently used measures [51,52] are considered such as magnitude relative error (MRE), mean of MRE (MMRE), root mean square error (RMSE), and prediction within 25% of the actual value. The definitions of the measures are discussed below.

- The magnitude of relative error (MRE) is calculated as given below.

$$MRE = \frac{ActualResponseTime - EstimatedResponseTime}{ActualResponseTime} \tag{2}$$

Table 5 Dataset of microservices based applications from literature

S.No	Name of application	#Modules	#User requests	Response time (ms)
1	Sock Shop application [42]	9	500	425.51
2	eShopper [43]	9	1000	666.5
3	Train Ticket [43]	40	100	249.32
4	Stan's Robot Shop [44]	12	50	47.2
5	Web Application [45]	6	1051	934
6	e-Shop Application [46]	5	100	1219
7	fabric store information system [47]	5	300	3703.2
8	xyz application [48]	4	1100	300
9	Web Application [49]	7	1676	232.67
10	WebShop Application [50]	5	1500	321.645

- MMRE is the mean of the MREs of all the applications and it is used to evaluate the prediction performance of the model. The Mean of MRE is calculated as

$$MMRE = \frac{1}{n}(\sum_{i=1}^{n} MRE_i) \tag{3}$$

- The Root Mean Square Error (RMSE) measures the difference between the actual and estimated response times. It is used to calculate the standard deviation of the errors that arise when the proposed approach is applied to the datasets. RMSE is calculated as given below.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(ActualResponseTime - EstimatedResponseTime)^2} \tag{4}$$

- To check whether the predicted values are within m% of the actual values, we employ a measure, with m generally set to 25% in software engineering. The measure PRED(25) is calculated as

$$PRED(25) = \frac{k}{n} \tag{5}$$

where k is the number of observations for those whose MRE is less than or equal to 0.25 and n is the total number of applications.

- Because the previously discussed measures are critiqued and react differently when evaluating prediction models, two additional measures are proposed [23]. The mean absolute error (MAE) is unbiased and is calculated as follows.

$$MAE = \frac{1}{n}\sum_{i=1}^{n} | ActualResponseTime_i - EstimatedResponseTime_i | \tag{6}$$

where n is the number of applications chosen for evaluation of performance.

- Standardised accuracy (SA) is one of the preferred measurements for comparing the performance of MAE-based prediction models. It is defined as follows:

$$SA = 1 - \frac{MAE_{Pj}}{MAE_{guess}} \times 100 \tag{7}$$

where MAE_{Pj} is the MAE of the proposed approach and MAE_{guess} is the value of MAE of a random guess. The standard accuracy represents the impact of MAE_{Pj} when compared to any random guess. For performance prediction techniques, the value of MAE should be less and SA should be maximum.

6.4 Results

The proposed regression method is used and the function for response time prediction is obtained. First, the variables x_1 and x_2 from the dataset table are analysed, and their values are used to calculate the response time using the regression model. To increase accuracy, we use a Leave-N-Out policy, in which we train the model only on the first eight applications to calculate the coefficients and then test it on the remaining two applications. The estimated coefficient values are shown in the response time prediction function, which is given in the equation below.

$$y = 1944.5075 - 47.8322x_1 - 0.8573x_2 \tag{8}$$

Using the calculated coefficient values β_0 = 1944.5075, β_1 = -47.8322, and β_2 = -0.8573, we test the remaining two applications. The response times calculated using the generated coefficients are presented in the Table 6. Due to lack of application data for different microservices, the proposed model does not generate better accuracy values. In machine learning, datasets play a key role in improving the accuracy of prediction

Table 6 Application of regression model on test data

S.No	Actual RT (ms)	Predicted RT (ms)	MRE
9	232.67	183.8473	0.209
10	321.645	411.3965	-0.279

Table 7 Accuracy of the prediction model

Model	MMRE	RMSE	PRED(25)	MAE	SA
Proposed model	0.348	719.125	0.5	332.25	66.75

and classification problems. The proposed regression model works as recommendation system for estimating the response time required for processing the user requests given to microservices.

6.5 Performance of Proposed Regression Model

The accuracy of the proposed regression model is evaluated using the metrics presented in Section 6.3 and the values are presented in Table 7. From the results, SA value is 66.75% and as stated before, due to lack of dataset of microservices applications, the accuracy of the models is less. However, the accuracy can be improved by applying optimization techniques such as gradient descent to form the best fit regression model for the chosen dataset applications.

6.6 Comparison

The proposed prediction function is computed using first eight applications by applying regression analysis leaving one application in each iteration. The values of coefficients are computed and the response times are generated for each application. The response times by the proposed regression model and the actual response times are compared and presented as line graph as shown in Figure 19. The x-axis represents the applications and y-axis represents the response times in milliseconds. From the graph, it is clear that the predicted response times are little closer to the actual response times of the applications.

7 Threats to Validity

The following threats to validity have been identified in the proposed work:

- The empirical evaluation carried out on the proposed framework is done on a case study application which is very small and has only four microservices. So, a detailed empirical analysis should be carried out on large applications or consider multiple applications for analysis to arrive at such a conclusion that microservices with microfrontends perform better.
- In the empirical investigation, the maximum workload given to the application is 10000 requests only due to lack of hardware and tool support. As in real-time, microservices based web applications receive

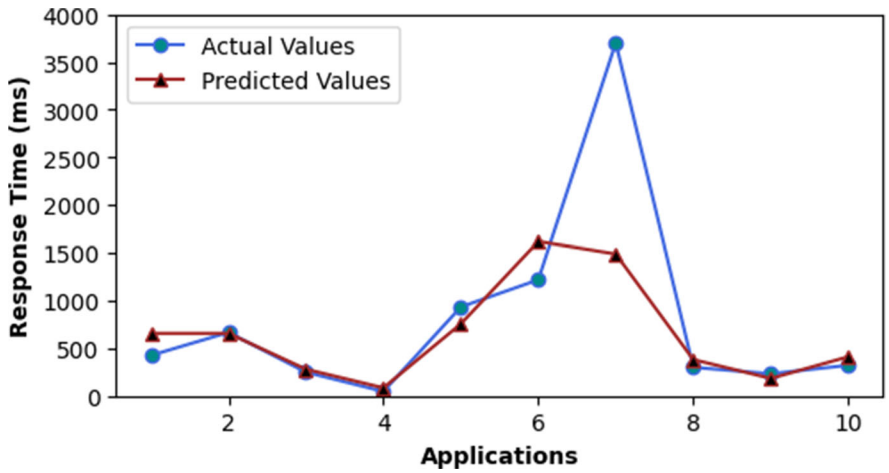


Fig. 19 Different measures for the proposed prediction model

thousands and lakhs of request, a thorough testing is required.
- Since the dataset related to performancfe of microservices is hard to find, the proposed prediction model is constructed using only ten applications. As observed from the results, the model framed is not accurate enough to claim. Hence, prediction model can be generated by using huge data only.

8 Conclusion

Performance is one of the very important QoS parameters for any software application as it has major impact on the business and the customer satisfaction. On the other hand, microservices architecture is trending and it has become a very popular style of designing large enterprise applications. Though microservices based application exhibit numerous benefits over other styles of designing applications, due to large number of services and complexity of the application, the performance of the applications can still be improved. In this paper, an attempt to improve the performance of microservices, micro-frontend based microservices application framework is proposed and comparative results are found to be satisfactory. Also, a regression model to predict the performance of the applications based on number of microservices in the application and the number of user requests given. A comparison of the proposed regression model with a multiple set of metrics is also presented. However, gathering datasets for microservices based application was a major challenge and hence, considering large datasets for applying the machine learning models can be proposed in future.

Declarations

Conflicts of interest The authors do not have any conflict of interest.

References

1. Lenarduzzi, V., Lomio, F., Saarimaki, N., Taibi, D.: Does migrating a monolithic system to microservices decrease the technical debt? J. Syst. Softw. **169**, 110710 (2020)
2. Kuryazov, D., Jabborov, D., Khujamuratov, B.: Towards decomposing monolithic applications into microservices. In 2020 IEEE 14th International Conference on Application of Information and Communication Technologies (AICT), pp. 1-4. IEEE, (2020)
3. Bucchiarone, A., Dragoni, N., Dustdar, S., Larsen, S.T., Mazzara, M.: From monolithic to microservices: An experience report from the banking domain. IEEE Softw. **35**(3), 50–55 (2018)
4. Ponce, F., Marquez, G., Astudillo, H.: Migrating from monolithic architecture to microservices: A Rapid Review. In 2019 38th International Conference of the Chilean Computer Science Society (SCCC), pp. 1-7. IEEE, (2019)
5. Raj, V., Ravichandra, S.: Microservices: A perfect SOA based solution for Enterprise Applications compared to Web Services. In 2018 3rd IEEE International Conference on recent trends in electronics, information & communication technology (RTEICT) pp. 1531-1536. IEEE (2018)
6. Raj, V., Sadam, R.: Performance and complexity comparison of service oriented architecture and microservices architecture. Int. J. Commun. Netw. Distrib. Syst. **27**(1), 100–117 (2021)
7. Al-Debagy, O., Martinek, P.: A comparative review of microservices and monolithic architectures. In 2018 IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI), pp. 000149-000154. IEEE, (2018)
8. Raj, V., Sadam, R.: Evaluation of SOA-based web services and microservices architecture using complexity metrics. SN Comput. Sci. **2**, 1–10 (2021)
9. Raj, V., Ravichandra, S.: A service graph based extraction of microservices from monolith services of service?oriented architecture. Softw. Pract. Exper. **52**(7), 1661–1678 (2022)
10. Taibi, D., Lenarduzzi, V., Pahl, C.: Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation. IEEE Cloud Comput. **4**(5), 22–32 (2017)
11. Abdullah, M., Iqbal, W., Berral, J.L., Polo, J., Carrera, D.: Burst-aware predictive autoscaling for containerized microservices. IEEE Trans. Serv. Comput. **15**(3), 1448–1460 (2020)
12. Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Li, Wenhai, Ding, Dan: Delta debugging microservice systems with parallel optimization. IEEE Trans. Serv. Comput. **15**(1), 16–29 (2019)
13. Wei, H., Rodriguez, J.S., Garcia, O.N.T.: Deployment management and topology discovery of microservice applications in the multicloud environment. J. Grid Comput. **19**, 1–22 (2021)
14. Magableh, Basel, Almiani, Muder: A deep recurrent Q network towards self?adapting distributed microservice architecture. Softw. Pract. Exper **50**(2), 116–135 (2020). Tomas Fernandez.: MicroFrontends: Microservices for the Frontend
15. https://semaphoreci.com/blog/microfrontends. Accessed 19-Feb-2023
16. Peltonen, S., Mezzalira, L., Taibi, D.: Motivations, benefits, and issues for adopting micro-frontends: a multivocal literature review. Inf. Softw. Technol. **136**, 106571 (2021)
17. Prajwal, Y., Parekh, J.V., Shettar, R.: A brief review of microfrontends. United Int. J. Res. Technol **2**, 123–126 (2021)
18. Joseph, C.T., Chandrasekaran, K.: IntMA: Dynamic Interaction-aware resource allocation for containerized

microservices in cloud environments. Journal of Systems Architecture **111**, 101785 (2020)
19. Sampaio, A.R., Rubin, J., Beschastnikh, I., Rosa, N.S.: Improving microservice-based applications with runtime placement adaptation. J. Int. Serv. Appl. **10**(1), 1–30 (2019)
20. ZargarAzad, M., Ashtiani, M.: An auto-scaling approach for microservices in cloud computing environments (2023)
21. Cortellessa, V., Di Pompeo, D., Eramo, R., Tucci, M.: A model-driven approach for continuous performance engineering in microservice-based systems. J. Syst. Softw. **183**, 111084 (2022)
22. Raj, V., Ravichandra, S.:Enhanced Service Point Approach for Microservices Based Applications Using Machine Learning Techniques. In International Conference on Advanced Informatics for Computing Research pp. 78-90. Cham: Springer International Publishing (2021)
23. Yang, C., Liu, C., Su, Z.: Research and application of micro frontends. In IOP conference series: materials science and engineering vol. 490, p. 062082. IOP Publishing (2019)
24. Prajwal, Y., Parekh, J. V., Shettar, R.: A brief review of micro-frontends. United Int. J. Res. Technol., 2(8) (2021)
25. Al Qassem, L.M., Stouraitis, T., Damiani, E., Elfadel, I.A.: Proactive Random-Forest Autoscaler for Microservice Resource Allocation. IEEE Access **11**, 2570–2585 (2023)
26. Yan, M., Liang, X., Lu, Z., Wu, J., Zhang, W.: HANSEL: Adaptive horizontal scaling of microservices using Bi-LSTM. Appl. Soft Comput. **105**, 107216 (2021)
27. Aydemir, F., Basciftci, F.: Building a Performance Efficient Core Banking System Based on the Microservices Architecture. J. Grid Comput. **20**(4), 37 (2022)
28. Noorabad, R., Charkari, N.M., Nogoorani, S.D.: PoMic: Dynamic Power Management of VM-Microservices in Overcommitted Cloud. J. Grid Comput. **21**(1), 12 (2023)
29. Taibi, D., Spillner, J., Wawruch, K.: Serverless computing-where are we now, and where are we heading? IEEE Softw. **38**(1), 25–31 (2020)
30. Raj, V., Sadam, R.: Patterns for Migration of SOA Based Applications to Microservices Architecture. J. Web Eng. **20**(5), 1229–1246 (2021)
31. Wei, H., Rodriguez, J.S., Garcia, O.N.T.: Deployment management and topology discovery of microservice applications in the multicloud environment. J. Grid Comput. **19**, 1–22 (2021)
32. Li, S., Zhang, H., Jia, Z., Li, Z., Zhang, C., Li, J., Gao, Q., Ge, J., Shan, Z.: A dataflow-driven approach to identifying microservices from monolithic applications. J. Syst. Softw. **157**, 110380 (2019)
33. Brondolin, R., Santambrogio, M.D.: A black-box monitoring approach to measure microservices runtime performance. ACM Trans. Archit. Code Optim. (TACO) **17**(4), 1–26 (2020)
34. Cinque, M., Della Corte, R., Pecchia, A.: Microservices monitoring with event logs and black box execution tracing. IEEE Trans. Serv. Comput. (2019)
35. Vayghan, L.A., Saied, M.A., Toeroe, M., Khendek, F.: A Kubernetes controller for managing the availability of elastic microservice based stateful applications. J. Syst. Softw. **175**, 110924 (2021)
36. Yan, M., Liang, X., Lu, Z., Wu, J., Zhang, W.: HANSEL: adaptive horizontal scaling of microservices using Bi-LSTM. Appl. Soft Comput. **105**, 107216 (2021)
37. Srirama, S.N., Adhikari, M., Paul, S.: Application deployment using containers with auto-scaling for microservices in cloud environment. J. Netw. Comput. Appl. **160**, 102629 (2020)
38. Wei, H., Rodriguez, J.S., Garcia, O.N.T.: Deployment management and topology discovery of microservice applications in the multicloud environment. J. Grid Comput. **19**(1), 1–22 (2021)
39. Brondolin, R., Santambrogio, M.D.: A black-box monitoring approach to measure microservices runtime performance. ACM Trans. Archit. Code Optim. (TACO) **17**(4), 1–26 (2020)
40. Wang, D., Yang, D., Zhou, H., Wang, Y., Hong, D., Dong, Q., Song, S.: A novel application of educational management information system based on micro frontends. Procedia Comput. Sci. **176**, 1567–1576 (2020)
41. Cully, K.: Performance of Microservices Result Data. (2021) https://dx.doi.org/10.21227/hhf7-8b30
42. Joseph, C.T., Chandrasekaran, K.: IntMA: Dynamic interaction-aware resource allocation for containerized microservices in cloud environments. J. Syst. Archit. **111**, 101785 (2020)
43. Cortellessa, V., Di Pompeo, D., Eramo, R., Tucci, M.: A model-driven approach for continuous performance engineering in microservice-based systems. J. Syst. Softw. **183**, 111084 (2022)
44. Xu, M., Song, C., Ilager, S., Gill, S.S., Zhao, J., Ye, K., Xu, C.: CoScal: Multifaceted scaling of microservices with reinforcement learning. IEEE Trans. Netw. Serv. Manag. **19**(4), 3995–4009 (2022)
45. Saransig, A., Tapia, F.: Performance analysis of monolithic and micro service architectures?containers technology. In Trends and Applications in Software Engineering: Proceedings of the 7th International Conference on Software Process Improvement (CIMPS 2018) 7 pp. 270-279. (2019) Springer International Publishing
46. Shabani, I., Meziu, E., Berisha, B., Biba, T.: Design of modern distributed systems based on microservices architecture. Int. J. Adv. Comput. Sci. Appl., 12(2) (2021)
47. Rahmatulloh, A., Nugraha, F., Gunawan, R., Darmawan, I.: November. Event-Driven Architecture to Improve Performance and Scalability in Microservices-Based Systems. In 2022 International Conference Advancement in Data Science, E-learning and Information Systems (ICADEIS) pp. 01-06. IEEE (2022)
48. Villamizar, M., Garces, O., Castro, H., Verano, M., Salamanca, L., Casallas, R., Gil, S.: September. Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud. In 2015 10th Computing Colombian Conference (10CCC) pp. 583-590. IEEE (2015)
49. Salunkhe, P.S.: Microservices vs Monolithic Architecture: Load Testing in AWS on ReactJS Web Application for Performance (Doctoral dissertation, Dublin, National College of Ireland) (2022)

50. Barczak, A., Barczak, P.M., Toledo, M.:Performance comparison of monolith and microservices based applications
51. Sarro, F., Petrozziello, A. and Harman, M.: Multi-objective software effort estimation. In Proceedings of the 38th International Conference on Software Engineering pp. 619-630 (2016)
52. Menzies, T., Yang, Y., Mathew, G., Boehm, B., Hihn, J.: Negative results for software effort estimation. Empir. Softw. Eng. **22**, 2658–2683 (2017)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.